Chapter 2: Variables, Statements and Expressions - Environment Setup & Core Syntax Essentials (Practical Foundation)

Introduction to Python: The Developer's Story The Story: How Guido van Rossum "Found" Python

In December 1989, a Dutch programmer named **Guido van Rossum** was looking for a hobby coding project to keep himself busy during the week around Christmas. He worked at a research lab in the Netherlands and wanted to create a new script-focused language that would bridge the gap between basic shell terminal scripts and complex C programming.

He decided to name the language Python. Contrary to popular belief, it wasn't named after the snake—Guido was a massive fan of the **British comedy television show Monty Python's Flying Circus**, and he wanted a name that felt slightly Quirky, short, and memorable. He published the code to the public in 1991, and it grew from a solo holiday project into the world's most dominant programming language.

Why Python? Features, Uses, Pros & Cons

1. Core Structural Features

- Human-Readable Syntax: Python reads almost like plain English. It strips away complex symbols like semicolons (;) and curly brackets ({}), relying entirely on clean spacing.
- Interpreted Execution: Python code is processed line-by-line at runtime by the interpreter. You do not need an extra compilation step before running your code.
- Cross-Platform Portability: The exact same Python file can run seamlessly on Windows, macOS, Linux, or Raspberry Pi microcomputers without modifying the code.

2. Major Real-World Applications

Python is not just a learning language; it runs the core architecture of tech giants like Google, Netflix, Instagram, and NASA.

- Data Science & Machine Learning: Building AI models, deep neural networks, and parsing massive scientific datasets.
- Web Development: Powering backend servers and web applications using frameworks like Django and Flask.
- Automation & Scripting: Writing Quick programs to automate repetitive computer tasks, parse files, or scrape data from websites.

3. Balanced Evaluation: Pros vs. Cons

Advantages (Pros)	Disadvantages (Cons)
Incredibly Easy to Learn: Friendly layout makes it perfect for absolute beginners.	Slower Execution Speed: Being an interpreted language, it runs slower than compiled languages like C++ or Java.
Massive Ecosystem: Millions of pre-written code packages (libraries) exist so you don't have to code features from scratch.	Weak for Mobile Apps: Python is rarely used to build native iOS or Android smartphone applications.
High Developer Demand: It is consistently ranked among the most sought-after coding skills in the tech job market.	High Memory Consumption: Its dynamic typing system and automatic cleanup features consume more RAM during heavy tasks.

Topic 1: Installing Python & Setting up the IDE

1. Architectural Blueprint (Theory & Syntax)

Python is an interpreted language, meaning it requires an interpreter to read and execute code line-by-line. When you install Python, you install this interpreter engine.

> *To write code efficiently, developers use an Integrated Development Environment (IDE) or a specialized Code Editor (like VS Code). An IDE provides syntax highlighting (color-coding keywords), auto-completion, and an integrated terminal to run scripts instantly.*

2. Live Walkthrough & Practical Code

Follow these steps live to verify your environment setup.

- 1.Download & Install Python.Go to python.org. Download the latest installer for your OS. Crucial: On Windows, you must check the box that says "Add python.exe to PATH" before clicking Install. If missed, the terminal will not recognize Python commands.
- 2.Verify Installation Open your terminal (Command Prompt on Windows, Terminal on Mac) and type the command below to ensure the system detects the interpreter.
- 3.Open the Editor. Launch Visual Studio Code (VS Code) or the built-in IDLE editor. Create a new file named main.py. The .py extension tells the system this is a Python executable script.

```
# Run this command in your system terminal to verify Python is working:

python --version

# Expected output should be similar to:

# Python 3.12.x (or higher)
```

```
# Verify your editor works by saving this single line in 'main.py' and running it:

print("Environment is fully configured!")
```

3. Verification & Diagnostic Exercises

Q1. Fill in the Blank

When installing Python on Windows, you must check the box named “Add python.exe to _____” so the terminal can locate the Python engine.

Q2. Syntax Correction

A student creates a script named program.txt and writes Python code inside it. When they try to run it via the terminal, it fails. What mistake did they make with the filename?

Q3. Identify the Tool

Which software component is responsible for reading Python code line-by-line and converting it into instructions the computer’s CPU can execute?

Q4. Code Execution Trace

If you run python --version in your terminal and see Command ‘python’ not found, what went wrong during the installation process?

Q5. Fill in the Blank

The file extension used for all standard Python script files is _____.

Q6. Conceptual Short Answer

What is the primary difference between a basic text editor (like Notepad) and an IDE like VS Code when writing code?

Topic 2: The print() Function & Writing Your First Script

1. Architectural Blueprint (Theory & Syntax)

> *The print() function sends data out of your program into the system terminal or console window. Execution flows sequentially (top-to-bottom, line-by-line).*

Syntax Pattern

```
print(value1, value2, ..., sep=' ', end='n')
```

- Multiple Values: Separated by commas inside the parentheses.
- sep: Controls what goes between values (defaults to a space " ").
- end: Controls what happens at the end of the line (defaults to a newline “n”).

2. Live Walkthrough & Practical Code

```
# 1. Standard single value output

print("Hello, Python Learners!")

# 2. Outputting multiple values (Python automatically separates them with spaces)

print("Welcome to", "Class", 101)

# 3. Customizing the separator using the sep parameter

print("Apple", "Banana", "Cherry", sep=" -> ")

# 4. Modifying the end parameter to prevent jumping to a new line

print("Loading", end="... ")

print("Complete!")

# 5. Understanding SeQuential Flow (Line-by-Line Execution)

print("Line 1: Starting the engine...")

print("Line 2: Processing core data...")

print("Line 3: Shutting down.")
```

> *Output of the script above:*

```
Output:

Hello, Python Learners!

Welcome to Class 101

Apple -> Banana -> Cherry

Loading... Complete!

Line 1: Starting the engine...

Line 2: Processing core data...

Line 3: Shutting down.
```

3. Verification & Diagnostic Exercises

Q1. Find the Output

```
print("Python", "Is", "Fun", sep="#")
```

Q2. Find the Output

```
print("Hello", end=" ")

print("World")
```

Q3. Error Identification

Identify the syntax error in the following code block:

```
print("Welcome to programming"

print("Let's build a script!")
```

Q4. Code Completion

Fill in the missing parameter so that the three words output as Red-Green-Blue:

```
print("Red", "Green", "Blue", _________)
```

Q5. Trace the Flow

Re-arrange these lines so they print out a logical execution se#### Quence:

```
print("Step 3: Eat the sandwich")

print("Step 1: Get two slices of bread")

print("Step 2: Spread peanut butter")
```

Q6. Output Prediction

```


print("A", "B", sep="", end="")

print("C")
```

Q7. Syntax Correction

Fix the Quotes in this script to make it valid:

```
print('Python says: "Hello world!")
```

Topic 3: Variables, Assignment, and Dynamic Typing

1. Architectural Blueprint (Theory & Syntax)

> *A variable is a named storage location in your computer's memory. In Python, you create a variable the exact moment you assign a value to it using the = assignment operator.*

> *Identifiers Naming Rules :*

- 1. Variables names should not start with numbers.
- 2. They must start with a lower or upper letter, or a special characters like _(underscore).
- 3. Python is case sensitive.

> *Python variable names cannot start with a number or contain special characters because it would introduce syntactic ambiguity, making it impossible for the interpreter to parse the code efficiently.*

When you run a script, Python's lexer reads your code character by character to break it down into meaningful tokens (like variables, numbers, or operators). Enforcing strict naming rules prevents collisions during this process.

> *Special characters (like +, -, *, /, @, $, %, and spaces) are strictly reserved as operators, delimiters, or structural syntax within the language.*

> *Comments In python can be written as Below:*

```
# This is a single line comment

'''This is a multi line comment
this comment has 2 lines '''
```

```
# Syntax Pattern for variables

variable_name = value
```

> *Python uses Dynamic Typing. This means you do not have to declare what kind of data (text, numbers) a variable holds ahead of time. The Python engine figures out the type automatically at runtime, and you can freely change that data type later by assigning a new value.*

2. Live Walkthrough & Practical Code

```
# 1. Variable Assignment

score = 100

player_name = "Alex"

print("Initial Profile:")

print(player_name, score)

# 2. Re-assignment (Updating values in memory)

score = 150

print("Updated Score:", score)

# 3. Dynamic Typing Demonstration

# 'data_holder' starts as an integer (whole number)


data_holder = 42

print("Type 1:", data_holder)

# 'data_holder' changes completely into a string (text block)

data_holder = "Now I am text!"

print("Type 2:", data_holder)

# 4. Assigning one variable's value to another

alpha = 5

beta = alpha

alpha = 20 # Changing alpha does NOT change beta

print("Alpha:", alpha, "Beta:", beta)
```

```
Output of the script above:

Initial Profile:

Alex 100

Updated Score: 150

Type 1: 42

Type 2: Now I am text!

Alpha: 20 Beta: 5
```

3. Verification & Diagnostic Exercises

Q1. Find the Output

```
x = 10

y = 20

x = y

print(x, y)
```

Q2. Error Identification

What rule does this variable assignment break?

```
77_lucky_number = 77
```

Q3. Fill in the Blank

Python's ability to change a variable's data type on the fly during program execution is called _____ typing.

Q4. Code Completion

Complete the code below so that the value of apple_count increases by 5:

```
apple_count = 10

apple_count = _________ + 5

print(apple_count)
```

Q5. True or False

The expression a = 5 is mathematically identical to 5 = a in Python.

Q6. Find the Output

```
val = "Green"

val = "Gold"

val = "Blue"

print(val)
```

Q7. Variable Swapping Output

```
m = 1
n = 2
temp = m
m = n
n = temp

print(m, n)
```

Topic 4: The Strict Rules of Python Indentation

1. Architectural Blueprint (Theory & Syntax)

> *Unlike languages like Java or C++ that use curly braces {} to group code blocks, Python uses structural indentation (whitespace spaces at the start of a line).*

- All code statements within the same block must be indented by the exact same number of spaces.
- The standard practice is 4 spaces per indentation level.
- Mixing spaces and tabs or varying the space count will trigger an IndentationError.

2. Live Walkthrough & Practical Code

```
# Correct Code Formatting (Standard Execution Flow)

print("This line runs at the baseline (level 0).")

print("This line also runs at the baseline.")

# A colon ':' always introduces a new indented block of code

if True:

    print("Inside block 1: This line has 4 spaces of indentation.")
    print("Inside block 1: Still matching perfectly with 4 spaces.")


    if True:
        print("Inside block 2: Indented by 8 spaces deep!")

print("Back to the baseline (level 0).")
```

Now let's look at code that causes immediate crashes due to bad indentation formatting:

```
# CRASH CODE 1: Spontaneous Indentation

print("Hello")

print("World") # Triggers: IndentationError: unexpected indent


# CRASH CODE 2: Mismatched Block Spaces

if True:

print("Step A")

print("Step B") # Triggers: IndentationError: unindent does not match any outer indent
```

3. Verification & Diagnostic Exercises

Q1. Identify the Error Line

Which line number will cause a crash when this file executes?

```
# Line 1

print("System Check initiated.")

# Line 2

print("Loading databases...")

# Line 3

print'("System Check complete.")'
```

Q2. True or False

Python allows you to use both tabs and spaces interchangeably in the same code block as long as they look aligned on your screen.

Q3. Error Diagnostics

What specific type of error will Python throw if your indentation alignment is broken?

Q4. Code Repair

Fix the indentation layout of this script so it runs without crashing:

```
if True:

print("Action 1")

print("Action 2")
```

Q5. Find the Output

```
if True:
    print("Red")

print("Blue")
```

Q6. Conceptual Short Answer

How does Python determine where a block of code starts and ends?

Q7. Fill in the Blank

The standard number of spaces recommended by Python documentation (PEP 8) for each level of indentation is ____.

Topic 5: Handling User Input with input()

1. Architectural Blueprint (Theory & Syntax)

> *The input() function pauses your script's execution and waits for the user to type something into the console terminal and press Enter.*

```
# Syntax Pattern

variable_to_store_data = input("Optional Prompt Message String")
```

- ***Critical Operational Rule***: The input() function always captures and returns data as a String (text block). It does not matter if the user types the number 45 or the word hello; Python reads it strictly as text symbols ("45" or "hello").

2. Live Walkthrough & Practical Code

```
# 1. Simple text capture

print("--- User Registration ---")

user_name = input("Enter your username: ")

print("Welcome aboard,", user_name)

# 2. Proving that numbers are captured as text strings

fav_number = input("Type your favorite number: ")

print("The data type you entered is actually:", type(fav_number))

# 3. The Danger of Text Concatenation (Gluing strings instead of doing math)


# If a user types 5 and 5, this will output 55, NOT 10!

num1 = input("Enter first number: ")

num2 = input("Enter second number: ")

result = num1 + num2

print("Warning! Merged string result is:", result)
```

```
Console Session Example Interaction:

--- User Registration ---

Enter your username: DevSarah

Welcome aboard, DevSarah

Type your favorite number: 7

The data type you entered is actually: <class 'str'>

Enter first number: 5

Enter second number: 5

Warning! Merged string result is: 55
```

3. Verification & Diagnostic Exercises

Q1. Find the Output

If the user inputs the digits 99 when prompted, what is the output?

```
user_age = input("Enter your age: ")

print(user_age + " years old")
```

Q2. Code Completion

Fill in the blank to prompt the user with the message "Enter color: ":

```
chosen_color = input(_________________)
```

Q3. Trace and Predict

If the user enters Jack on the first prompt and Frost on the second prompt, what prints?

```
first = input()

last = input()

print(last, first)
```

Q4. Conceptual Short Answer

Why does input() create unexpected results if you try to immediately perform math operations on the data it returns?

Q5. True or False

The input() function will automatically crash if the user presses Enter without typing any text characters.

Q6. Find the Output

If the user types Hello when the program runs:

```
captured = input("Say something: ")

print(captured, captured, sep="-")
```

Q7. Syntax Correction

Fix the structural syntax error in this input line:

```
data = input(Enter values)
```

Topic 6: Introduction to Core Data Types

1. Architectural Blueprint (Theory & Syntax)

> *Data types define what kind of value a variable holds, determining what operations can be performed on it. Python categorizes its most basic primitive data types as follows:*

Data Type Name	Python Code Keyword	Representation / Purpose	Example Syntax
Integer	int	Whole numbers (positive, negative, zero)	45, -700
Floating Point	float	Fractional decimal numbers	3.1415, -0.004

> *Some More Data Types:*

Data Type Name	Python Code Keyword	Representation / Purpose	Example Syntax
String	str	Textual data enclosed in matching #### Quotes	“Hello”, ‘Code’

Data Type Name	Python Code Keyword	Representation / Purpose	Example Syntax
Boolean	bool	Logical truth states	True, False

You can check the structural data type of any variable at any point by passing it to the built-in type() function.

2. Live Walkthrough & Practical Code

```
# Assigning values across all four primary core data types

age = 19

# Integer (int)

price = 99.95 # Floating Point (float)

course_name = "Python" # String (str)

is_active = True # Boolean (bool) - Case sensitive capital T!

# Displaying values alongside their evaluated internal types

print("Age value:", age, "-> Type:", type(age))

print("Price value:", price, "-> Type:", type(price))

print("Course value:", course_name, "-> Type:", type(course_name))

print("Status value:", is_active, "-> Type:", type(is_active))

# Demonstrating type distinctions (An int vs a str containing a digit)

print("Are mystery_a and mystery_b the same type?")

print(type(mystery_a) == type(mystery_b))
```

```
Output of the script above:

Plaintext

Age value: 19 -> Type: <class 'int'>

Price value: 99.95 -> Type: <class 'float'>

Course value: Python -> Type: <class 'str'>

Status value: True -> Type: <class 'bool'>

Are mystery_a and mystery_b the same type?

False
```

3. Verification & Diagnostic Exercises

Q1. Identify the Data Type

What is the exact internal data type keyword for the value assigned here?

```
speed = 55.0
```

Q2. Find the Output

```
print(type("True"))
```

Q3. Error Identification

Why does this specific assignment line cause a script crash when parsed?

```
is_weekend = true
```

Q4. Code Completion

Fill in the blanks to verify the type of the value -999:

```
val = -999

print(________(val))
```

Q5. Factual Matching

Match the literal values below to their matching data types (int, float, str, bool):

“3.14” -> _____

3.14 -> _____

3 -> _____

Q6. Predict the Output

```
x = 100

print(type(x) == int)
```

Q7. True or False

A string type variable can only be wrapped in double-Quotes " ", and using single- Quotes ’ ’ will cause an immediate crash.

Topic 7: Type Conversion / Typecasting Basics

1. Architectural Blueprint (Theory & Syntax)

> *Type conversion (also known as typecasting) is the explicit process of forcing a variable to change from one data type to another.*

Since the input() function only outputs text data (str), you must cast that data into a numeric type (int or float) if you plan on doing math.

```
# Core Typecasting Functions

int(value) # Converts value to an Integer

float(value) # Converts value to a Float

str(value) # Converts value to a String
```

- ***Operational Limit***: If you try to cast a string that contains non-numeric characters (like “apple” or “12a3”) into an integer, Python will throw a ValueError.

2. Live Walkthrough & Practical Code

```python
# 1. The Right Way to process numeric terminal inputs
user_input = input("Enter your birth year: ") # returns e.g. "2005" (str)
birth_year = int(user_input)
# explicitly cast to integer
current_year = 2026

calculated_age = current_year - birth_year
print("You are roughly:", calculated_age, "years old.")
# 2. Converting integers to floats, and floats to integers
base_val = 10
float_version = float(base_val)
print("Int to Float Conversion:", float_version)
pi_estimate = 3.99
truncated_int = int(pi_estimate) # Note: Int conversion drops everything after decimal
print("Float to Int Truncation:", truncated_int)
# 3. Demonstration of a Typecasting Crash (Unconvertible Data)
try:
    bad_conversion = int("Python123")
except ValueError:
    print("Caught Error: You cannot convert letters into pure numbers!")
```

```
Output of the script above:

Plaintext

Enter your birth year: 2005

You are roughly: 21 years old.

Int to Float Conversion: 10.0

Float to Int Truncation: 3

Caught Error: You cannot convert letters into pure numbers!
```

3. Verification & Diagnostic Exercises

Q1. Find the Output

```
a = "10"

b = "20"

print(int(a) + int(b))
```

Q2. Output Prediction

What prints out when this specific conversion runs?

```
print(int(5.85))
```

Q3. Error Diagnostics

What specific runtime error does Python raise if you execute int("hello")?

Q4. Code Completion

Fill in the missing typecasting function to successfully add the decimal value:

```
price_text = "19.99"

total = ________(price_text) + 5.00

print(total)
```

Q5. Find the Output

```
value = 50

text_value = str(value)

print(text_value + "0")
```

Q6. True or False

Converting a float to an integer via int() will round the value to the nearest whole number.

Q7. Code Fix

Fix the code block below so that it performs numerical addition rather than joining strings:

```
x = input("Num 1: ")

y = input("Num 2: ")

print(x + y)
```

Topic 8: Basic Arithmetic Expressions & Evaluation

1. Architectural Blueprint (Theory & Syntax)

Arithmetic expressions combine numeric values and operational operators (+, -, *, /) to calculate a single final output value.

> *Python follows standard mathematical order of operations (PEMDAS: Parentheses, Exponents, Multiplication/Division, Addition/Subtraction).*

Operator Symbol	Math Operation Name	Operational Behavior Example
+	Addition	5 + 3 evaluates to 8
-	Subtraction	10 - 4 evaluates to 6
*	Multiplication	3 * 4 evaluates to 12
/	True Division	7 / 2 evaluates to 3.5 (Always yields a float!)

2. Live Walkthrough & Practical Code

This script puts all the concepts from Topics 1 through 7 together into a working program:

a functioning terminal calculator that takes user input, casts it, performs math, and outputs the final result.

```
print("=======================================")

print(" MINI-CALCULATOR PROGRAM ")

print("=======================================")

# Step 1: Capture interactive inputs from user

raw_num1 = input("Enter your first number: ")

raw_num2 = input("Enter your second number: ")

# Step 2: Use typecasting to convert text inputs into usable floats


number1 = float(raw_num1)

number2 = float(raw_num2)

# Step 3: Perform standard arithmetic evaluations

sum_result = number1 + number2

diff_result = number1 - number2

prod_result = number1 * number2

div_result = number1 / number2

# Step 4: Output calculations clearly to the console terminal

print("n--- Evaluation Results ---")

print("Addition Result ( + ) :", sum_result)

print("Subtraction Result ( - ) :", diff_result)

print("Multiplication Result (* ):", prod_result)

print("Division Result ( / ) :", div_result)

print("=======================================")
```

```
Console Session Example Interaction:

Plaintext

========================================

MINI-CALCULATOR PROGRAM

========================================

Enter your first number: 12

Enter your second number: 4

--- Evaluation Results ---

Addition Result ( + ) : 16.0

Subtraction Result ( - ) : 8.0

Multiplication Result (* ): 48.0


Division Result ( / ) : 3.0

========================================
```

3. Verification & Diagnostic Exercises

Q1. Find the Output

```
result = 10 + 2 * 5

print(result)
```

Q2. Order of Operations Evaluation

```
val = (10 + 2) * 5

print(val)
```

Q3. Find the Output Type

What data type is the final output of the code below?

```
print(type(10 / 2))
```

Q4. Code Repair

Identify and correct the calculation error in this average-finding program:

```
num1 = 10

num2 = 20
```

```
# Find the average of both numbers

average = num1 + num2 / 2

print(average)
```

Q5. Fill in the Blank

In Python arithmetic, the true division operator (/) always outputs its final answer as a _____ data type, even if the numbers divide perfectly.

Q6. Output Prediction

```
x = 5

y = 2

print(x * y + x)
```

Q7. Complex Expression Evaluation

```
calc = 20 - 5 * 2 + 8 / 4

print(calc)
```

Some More Codes to Crack !!

1. Output Prediction (Data Types & Precision)

```python
# Question 1: Predict the exact output and data types

a = 10 / 2

b = 10 // 3

c = 2 ** 3 ** 2

print(f"a: {a} ({type(a)})")

print(f"b: {b} ({type(b)})")

print(f"c: {c}")
```

> *Standard division / always returns a float, floor division // truncates the decimal, and exponentiation evaluates right-toleft ($3^2 = 9$, then $2^9 = 512$).*

2. Output Prediction (String Replication vs. Addition)

```python
# Question 2: Predict the output

x = "Python"

y = 3

print(x * y)

print(x + str(y))
```

> *String replication using * versus string concatenation using +, highlighting why mixing types without str() throws a TypeError.*

3. Error Finding (Immutability Violation)

```python
# Question 3: Find the error in this snippet

msg = "hello world"

msg[0] = "H"

print(msg)
```

4. Error Finding (Variable Naming & Tokenization)

```
# Question 4: Identify the syntax violation

2nd_user = "Alice"

class = "Data Science"

user-name = "Bob"
```

5. Coding Challenge (Swap Two Variables Without Temp)

```
# Question 5: Swap the values of p and q without using a third variable

p = 15

q = 99

# Write your code here:

print(f"p: {p}, q: {q}")
```

6. Coding Challenge (Dynamic Type Casting Evaluator)

```
# Question 6: Accept a user input, convert it to float, multiply by 5, and print it as

data = input("Enter a number: ")

# Write the conversion logic here:
```

7. Fill in the Blanks (Type Checking Syntax)

```
# Question 7: Fill in the blanks to verify if 'val' is an integer

val = 45.5

# if ___________(val, _______):

#     print("It's an integer!")

# else:

#     print("It's not an integer!")
```

8. Fill in the Blanks (Memory Management Validation)

```
# Question 8: Fill in the blank to print the actual memory address of the object

course = "Python Masterclass"

print("Memory ID is:", __________(course))
```

9. True or False (Dynamic Typing)

```
# Question 9: True or False?

# "In Python, once a variable is declared as an integer,

- #  it cannot store a string value later in the script."

score = 100

score = "Excellent"

print("Executed successfully without crashing?")
```

10. True or False (Boolean Evaluations of Falsy Values)

```
# Question 10: True or False?

# "Empty collections and the number 0 are evaluated as False in a boolean context."

print(bool(0))

print(bool(""))

print(bool([]))
```

11. Fill in the Blanks (Formatted String Literals)

```
# Question 11: Fill in the blanks to round the value of pi to 3 decimal places

pi_val = 3.14159265

# print(f"Pi rounded: {pi_val:________}")
```

12. Error Finding (Implicit Casting Boundaries)**

```
# Question 12: Identify why this code crashes and fix the dynamic concatenation

user_age = 21

message = "Access granted. You are " + user_age + " years old."

print(message)
```

13. Output Prediction (Escape Sequences & Raw Strings)**

```
# Question 13: Predict the exact output configuration of these two paths

path_a = "C:\new_folder\test.py"

path_b = r"C:\new_folder\test.py"

print("Path A:", path_a)
print("Path B:", path_b)
```